

COMENIUS UNIVERSITY IN BRATISLAVA
FACULTY OF MATHEMATICS, PHYSICS AND INFORMATICS

THE USE OF AI IN THE DIGITAL
TRANSFORMATION OF GOVERNMENT OR
PUBLIC ADMINISTRATION PROCESSES.
BACHELOR THESIS

2026
DANIEL KOTULÁČ

COMENIUS UNIVERSITY IN BRATISLAVA
FACULTY OF MATHEMATICS, PHYSICS AND INFORMATICS

THE USE OF AI IN THE DIGITAL
TRANSFORMATION OF GOVERNMENT OR
PUBLIC ADMINISTRATION PROCESSES.
BACHELOR THESIS

Study Programme: Computer Science
Field of Study: Computer Science
Department: Department of Computer Science
Supervisor: PhDr. Ing. Tomáš Gál, PhD.

Bratislava, 2026
Daniel Kotuláč



Univerzita Komenského v Bratislave
Fakulta matematiky, fyziky a informatiky

ZADANIE ZÁVEREČNEJ PRÁCE

Meno a priezvisko študenta: Daniel Kotuláč
Študijný program: aplikovaná informatika (Jednoodborové štúdium, bakalársky I. st., denná forma)
Študijný odbor: informatika
Typ záverečnej práce: bakalárska
Jazyk záverečnej práce: slovenský
Sekundárny jazyk: anglický

Názov: Využitie AI pri digitálnej transformácii procesov štátnej alebo verejnej správy
Application of Artificial Intelligence in the Digital Transformation of Public Administration Processes

Anotácia: Bakalárska práca skúma potenciál využitia existujúcich alebo rozvíjaných techník umelej inteligencie pri digitálnej transformácii procesov v štátnej a verejnej správe — identifikuje konkrétne vybrané oblasti (registry, workflow systémy, rozhodovacie procesy, automatizácia overovania údajov a pod.), v ktorých by AI mohla významne znížiť transakčné náklady, zefektívniť prevádzku a zvýšiť kvalitu služieb. Práca zároveň prihliada k netechnickým prekážkam implementácie takýchto riešení (právne, etické, technické), v relevantnej miere berie do úvahy riziká a vypracuje návrhy optimálneho nasadenia v kontexte slovenského verejného sektora.

Cieľ: Cieľom práce je identifikovať a analyzovať možnosti využitia umelej inteligencie pri digitálnej transformácii procesov verejnej správy s dôrazom na znižovanie transakčných nákladov a zvyšovanie efektívnosti služieb.

Kľúčové slová: umelá inteligencia, digitálna transformácia, verejná správa, štátne IT, transakčné náklady, efektívnosť, registre, automatizácia procesov

Vedúci: PhDr. Ing. Tomáš Gál, PhD.
Katedra: FMFI.KAI - Katedra aplikovanej informatiky
Vedúci katedry: doc. RNDr. Tatiana Jajcayová, PhD.
Dátum zadania: 29.09.2025

Dátum schválenia: 27.10.2025

doc. RNDr. Damas Gruska, PhD.
garant študijného programu

.....
študent

.....
vedúci práce



THESIS ASSIGNMENT

Name and Surname: Daniel Kotuláč
Study programme: Applied Computer Science (Single degree study, bachelor I. deg., full time form)
Field of Study: Computer Science
Type of Thesis: Bachelor's thesis
Language of Thesis: Slovak
Secondary language: English

Title: Využitie AI pri digitálnej transformácii procesov štátnej alebo verejnej správy
Application of Artificial Intelligence in the Digital Transformation of Public Administration Processes

Annotation: Bakalárska práca skúma potenciál využitia existujúcich alebo rozvíjaných techník umelej inteligencie pri digitálnej transformácii procesov v štátnej a verejnej správe — identifikuje konkrétne vybrané oblasti (registry, workflow systémy, rozhodovacie procesy, automatizácia overovania údajov a pod.), v ktorých by AI mohla významne znížiť transakčné náklady, zefektívniť prevádzku a zvýšiť kvalitu služieb. Práca zároveň prihliada k netechnickým prekážkam implementácie takýchto riešení (právne, etické, technické), v relevantnej miere berie do úvahy riziká a vypracuje návrhy optimálneho nasadenia v kontexte slovenského verejného sektora.

Aim: Cieľom práce je identifikovať a analyzovať možnosti využitia umelej inteligencie pri digitálnej transformácii procesov verejnej správy s dôrazom na znižovanie transakčných nákladov a zvyšovanie efektívnosti služieb.

Keywords: umelá inteligencia, digitálna transformácia, verejná správa, štátne IT, transakčné náklady, efektívnosť, registre, automatizácia procesov

Supervisor: PhDr. Ing. Tomáš Gál, PhD.
Department: FMFI.KAI - Department of Applied Informatics
Head of department: doc. RNDr. Tatiana Jajcayová, PhD.

Assigned: 29.09.2025

Approved: 27.10.2025 doc. RNDr. Damas Gruska, PhD.
Guarantor of Study Programme

Student

Supervisor

Čestné vyhlásenie

Čestne vyhlasujem, že celú prácu na tému *The use of AI in the digital transformation of government or public administration processes.*, vrátane všetkých jej príloh a obrázkov, som vypracoval samostatne, a to s použitím literatúry uvedenej v priloženom zozname a nástrojov umelej inteligencie.

Vyhlasujem, že nástroje umelej inteligencie som použil v súlade s príslušnými právnymi predpismi, akademickými právami a slobodami, etickými a morálnymi zásadami za súčasného dodržania akademickej integrity a že ich použitie je v práci vhodným spôsobom označené.

Abstrakt

Táto bakalárska práca sa zameriava na návrh, tréovanie a implementáciu automatizovaného softvérového systému na detekciu a mapovanie poškodení povrchu vozovky pomocou umelej inteligencie. Hlavným cieľom je poskytnúť cenovo dostupnú alternatívu k subjektívnemu manuálnemu nahlasovaniu a drahým LiDARovým meraniam s využitím bežných akčných kamier umiestnených na vozidlách samosprávy. Jadrom vizuálneho rozpoznávania je na mieru natrénovaný model detekcie objektov YOLO26, ktorý bol tréovaný na upravenej dátovej sade 1 369 obrázkov. Pre prepojenie týchto digitálnych detekcií s fyzickým svetom bol vyvinutý vlastný softvér v jazyku Python. Tento program obsahuje automatické dávkové spracovanie, deduplikačnú logiku založenú na maximálnej miere istoty a algoritmus časovej synchronizácie najbližšieho suseda, ktorý presne páruje video snímky s vysokofrekvenčnou GPS telemetriou.

Kvantitatívne vyhodnotenie modelu dosiahlo hodnotu strednej priemernej presnosti (mAP@0.5) 0,841 a maximálne F1 skóre 0,79. Následné testovanie v reálnej prevádzke preukázalo schopnosť softvéru spracovať súvislý videozáznam a autonómne vygenerovať interaktívnu geopriestorovú HTML mapu detegovaných výtlkov. Tieto výsledky potvrdzujú, že navrhnutá architektúra predstavuje funkčný koncept, ktorý poskytuje lokálnym samosprávam objektívny základ pre dátami riadenú správu cestnej infraštruktúry.

Kľúčové slová: umelá inteligencia, počítačové videnie, YOLO26, detekcia objektov, degradácia vozovky, priestorovo-časová synchronizácia, Smart City.

Abstract

This bachelor's thesis focuses on the design, training, and implementation of an automated software system for detecting and mapping road surface degradation using artificial intelligence. The primary objective is to provide a cost-effective alternative to subjective manual reporting and expensive LiDAR surveys by utilizing consumer-grade action cameras mounted on municipal vehicles. The core visual recognition is powered by a custom-trained YOLO26 object detection model, trained on a curated dataset of 1,369 images. To anchor these digital detections to the physical world, a custom Python pipeline was developed. This software features automated batch processing, peak-confidence deduplication logic, and a nearest-neighbor temporal synchronization algorithm that accurately pairs video frames with high-frequency GPS telemetry.

Quantitative evaluation of the model yielded a mean Average Precision (mAP@0.5) of 0.841 and a peak F1-score of 0.79. Subsequent real-world testing demonstrated the pipeline's capability to process continuous video footage and automatically output an interactive geospatial HTML map of the detected road defects. These findings indicate that the proposed architecture serves as a viable proof of concept, providing a functional baseline for data-driven road infrastructure management in municipal applications.

Keywords: Artificial Intelligence, Computer Vision, YOLO26, Object Detection, Road Degradation, Spatiotemporal Synchronization, Smart City.

Contents

Introduction	1
1 Theoretical Framework and Current State Analysis	5
1.1 Artificial Intelligence in Public Administration	5
1.2 Current State of AI in the Public Sector	5
1.2.1 Infrastructure and Operational Efficiency	6
1.2.2 Public Health and Policy Analysis	6
1.3 Challenges and Risks	7
1.4 Theoretical Foundations of the Proposed Technologies	7
1.4.1 Deep Learning and Computer Vision	7
1.4.2 Object Detection and the YOLO Paradigm	7
1.4.3 Spatiotemporal Data Fusion	8
1.5 Analysis of Road Infrastructure Monitoring	8
1.5.1 Manual Reporting and Citizen Crowdsourcing	8
1.5.2 Specialized Profiler Vehicles (LiDAR)	8
1.6 Comparative Analysis and Proposed Solution	9
2 System Specification and Objectives	11
2.1 Main Objectives	11
2.2 Functional Requirements	11
2.3 Non-Functional Requirements	12
2.4 Target Audience and Use Cases	13
3 System Architecture and Design	15
3.1 Data Acquisition and Recording	15
3.2 Telemetry Extraction Strategy	15
3.3 Dataset Construction and Model Training	16
3.3.1 Hybrid Dataset Aggregation	17
3.3.2 False Positive Mitigation	17
3.4 Object Tracking and Deduplication Logic	17
3.4.1 Sequential Object Tracking	17

3.4.2	Peak Confidence Deduplication	18
3.5	Spatiotemporal Synchronization	18
3.6	System Output Generation	19
4	Software Implementation and Development	21
4.1	Data Preprocessing and Environment Setup	21
4.2	Model Training and Hyperparameter Configuration	22
4.2.1	Data Leakage Anomaly and Retraining Strategy	22
4.2.2	Hyperparameter Tuning	23
4.2.3	False Positive Mitigation via Background Images	23
4.3	Inference and Sequential Object Tracking	23
4.4	Spatiotemporal Synchronization Algorithm	24
4.5	Geospatial Output and Visualization	25
5	Evaluation and Results	27
5.1	Evaluation Metrics	27
5.2	Confidence Threshold Analysis	28
5.3	Overall Performance and Precision-Recall Dynamics	29
5.4	Classification Accuracy and Error Analysis	30
5.5	Hardware Environment and Training Dynamics	31
5.6	Real-World Pipeline Evaluation	31
5.7	Summary of Results	33
	Conclusion	35
	Príloha A	39

Introduction

Even though English is the secondary language of this thesis, the text of this thesis is mostly in English due to the fact that most of the literature regarding this topic and the documentation for the software and tools used is in English, and it would be difficult to translate all of them properly.

The technological landscape of the 21st century has been characterized by rapid advancements in the development of information technology, with Artificial Intelligence (AI) and Machine Learning (ML) emerging as the most transformative forces of this era. For years, these advanced computational models were largely confined to academic research or the private sector, where they have been extensively utilized to optimize logistics, personalize consumer experiences, and maximize corporate efficiency. However, as the underlying technology has become more accessible and robust, its application is inevitably expanding into the public sector. Governments, municipalities, and state institutions worldwide are currently facing mounting pressure to undergo comprehensive digital transformations. They must operate under increasingly tight financial budgets while meeting the escalating expectations of citizens who, accustomed to the seamless digital services of the commercial sphere, now demand a similar level of speed, transparency, and personalization from their public institutions.

Recognizing the potential of using Artificial Intelligence to improve public services and management, I developed an interest in this topic. Consequently, during the previous academic year, I conducted an overview study to comprehensively map the use of AI in public sector management. In that preparatory work, I analyzed how various nations leverage AI to streamline bureaucratic workflows, enhance public health surveillance, and improve the accessibility of e-government services. While conducting that work, I evaluated both the profound benefits and the inherent risks, such as data privacy concerns and algorithmic bias—associated with deploying AI in governance.

It was during this research project that I encountered a particularly compelling case study from the Netherlands that inspired me to develop my own project to improve public services. The city of Amsterdam had successfully implemented an automated computer vision system designed to detect uncollected garbage and illegal dumping on its streets. By mounting standard cameras on municipal vehicles, such as garbage trucks and public transit, the city created a passive, continuous scanning

network. When the AI model detected discarded trash, it automatically extracted the geographical coordinates and alerted the waste management department, allowing them to dispatch cleaning crews immediately.

Inspired by this project, I decided to adapt this technological concept into my own applied software engineering project for this bachelor's thesis. I sought to identify a comparable, persistent issue that heavily burdens local municipalities, drains public maintenance budgets, and negatively impacts citizens on a daily basis. The clear choice was road infrastructure degradation, specifically the formation of road potholes.

The maintenance of road networks is a critical civic responsibility, yet the traditional methods of monitoring road conditions are inherently flawed. Currently, municipalities rely primarily on manual, physical inspections by road administrators, which are slow, labor-intensive, and financially burdensome. Alternatively, they depend on crowdsourced citizen reports (such as through the Waze application or local civic portals), which are subjective, inconsistent, and often inaccurate. While highly precise, specialized profiler vehicles equipped with LiDAR exist, their prohibitive costs make them entirely inaccessible for routine, high-frequency scanning by smaller cities and towns.

Therefore, the primary objective of this bachelor's thesis is to bridge this technological gap by designing, training, and implementing an automated, low-cost computer vision system capable of detecting and mapping road surface damage. Drawing direct inspiration from the Dutch model, this project utilizes standard, off-the-shelf action cameras (such as the GoPro series) that can be easily mounted on bicycles or municipal vehicles. By feeding the recorded video through a custom-trained deep Convolutional Neural Network based on the YOLO (You Only Look Once) architecture, the system is designed to automatically identify potholes and explicitly ignore visual anomalies like manhole covers. Furthermore, by developing a custom data-fusion algorithm, the system extracts embedded GPS telemetry from the video files and generates a clean, highly accurate map of the road defects.

This thesis serves as a tangible proof of concept, demonstrating how the theoretical advantages of artificial intelligence can be engineered into a practical, accessible software tool. This work is divided into six chapters. The first chapter provides the theoretical framework, summarizing my previous research on AI in the public sector and the current state of road monitoring. The second chapter defines the precise technical specifications and objectives of the software. The third chapter outlines the high-level system architecture and data flow. The fourth chapter, the engineering core of the thesis, details the practical implementation, the training of the YOLO26 model, the creation of a custom dataset, and the resolution of challenges with the detection pipeline. The fifth chapter evaluates the quantitative accuracy of the model and its qualitative performance on real-world video footage. Finally, the conclusion

summarizes the project's contributions and proposes avenues for future development and real-world deployment.

During the preparation of this thesis, the Gemini AI tool was used to prepare background materials, structure the text, revise the academic content, and provide programming consultations. The author bears full responsibility for the accuracy of the final text.

Chapter 1

Theoretical Framework and Current State Analysis

To establish a comprehensive theoretical foundation for integrating artificial intelligence (AI) into public administration, the following sections analyze the intersection of digital governance and specialized computer vision technologies. Building upon the necessity for digital transformation in the public sector, this chapter evaluates existing literature, case studies, and the technological limitations of current road monitoring methods. Finally, it introduces the specific theoretical principles of object detection and data fusion required to justify the practical solution proposed in this thesis.

1.1 Artificial Intelligence in Public Administration

Artificial intelligence in public administration involves the use of machine learning, data analytics, and automation tools to perform tasks that were previously executed exclusively by human civil servants. The primary goal is to modernize bureaucratic processes, increase the resilience of public services, and transition governments from a reactive posture to a proactive, data-driven methodology [1, 2].

In the context of "Smart Cities," AI acts as a critical layer of infrastructure management. Administrative efficiency is one of the main driving forces behind this integration. AI systems support public objectives by automating repetitive, high-volume tasks, thereby reducing bureaucratic delays and allowing human workers to focus on complex cases requiring empathy, nuance, and strategic judgment [3, 4].

1.2 Current State of AI in the Public Sector

The transition of artificial intelligence from theoretical computer science into practical public sector deployment represents a significant methodological development in mod-

ern governance. Public institutions worldwide are increasingly moving beyond basic digitalization toward autonomous, algorithmic decision-making. This transformation is primarily driven by the exponential growth of municipal data and the necessity to optimize limited public budgets. By deploying machine learning architectures, governments are beginning to solve complex logistical problems in real-time, providing practical models that directly influence the development of automated systems like the one proposed in this thesis.

1.2.1 Infrastructure and Operational Efficiency

A highly relevant example of applied computer vision in infrastructure management is the city of Amsterdam's *Object Detection Kit (ODK)* initiative. The municipality implemented an open-source AI platform specifically to detect uncollected garbage and illegal waste dumping across the city [5]. By mounting standard cameras on existing municipal vehicles (like garbage trucks), the city created a passive, high-frequency scanning network. When the object detection model identifies wrongly placed waste, the system automatically extracts the geographic coordinates and transmits an actionable report to the sanitation department. This specific model of passive data collection, leveraging existing fleets rather than purchasing specialized surveying vehicles, directly inspired the architectural approach to automated pothole detection adopted in this project.

1.2.2 Public Health and Policy Analysis

In the realm of public health and resource allocation, AI has proven capable of optimizing severely limited physical resources. A prominent example is the "Eva" system deployed by the Greek government during the COVID-19 pandemic [6]. Faced with a massive influx of summer tourists and a scarce supply of PCR tests, Greece implemented a reinforcement learning algorithm at its border checkpoints. The system dynamically analyzed demographic data and recent testing results to estimate the viral prevalence among asymptomatic travelers, effectively deciding who should be tested. Research published in *Nature* demonstrated that this AI-driven approach identified nearly twice as many infected travelers as conventional randomized testing protocols [6]. This demonstrates the core value of AI in public administration: maximizing the utility of limited state resources through data-driven targeting.

1.3 Challenges and Risks

Despite its demonstrated operational benefits, the adoption of AI brings complex challenges that must be rigorously addressed to maintain citizen trust and legal compliance [3, 2].

- **Data Privacy:** Implementation must comply with strict legal frameworks like the European GDPR to ensure that the sensitive data processed by AI is not misused or exposed to cyberattacks [1].
- **Algorithmic Bias:** AI systems are entirely dependent on their training data. If the historical data contains human prejudices or geographical blind spots, the AI may institutionalize and amplify them at scale [4].
- **The "Black Box" Problem:** Many advanced deep learning models are mathematically difficult to interpret. In the public sector, decisions must be legally justifiable, requiring the development and implementation of "Explainable AI" (XAI) [7].

1.4 Theoretical Foundations of the Proposed Technologies

To address municipal challenges efficiently, the practical implementation of automated monitoring relies on specific theoretical advancements in computer science. The following principles form the technical core of the proposed pothole detection system.

1.4.1 Deep Learning and Computer Vision

Computer vision is a field of artificial intelligence that trains computers to interpret and understand the visual world. Modern computer vision relies heavily on Deep Learning, specifically Convolutional Neural Networks (CNNs). Unlike traditional machine learning algorithms that require manual feature engineering by a human programmer, CNNs automatically learn spatial hierarchies of features from raw images. By passing image matrices through multiple mathematical convolutional layers, the network learns to identify basic edges in the early layers and highly complex objects, such as the specific degradation topology of an asphalt pothole, in the deeper layers [8].

1.4.2 Object Detection and the YOLO Paradigm

Within the realm of computer vision, object detection must simultaneously solve two theoretical problems: classification (identifying *what* an object is) and localization

(finding *where* it is, usually via a bounding box).

Object detection models are generally divided into two distinct paradigms:

- **Two-Stage Detectors:** (e.g., Faster R-CNN) These algorithms first propose regions of interest within an image and then classify them. While theoretically highly accurate, they are computationally heavy and slow.
- **Single-Stage Detectors:** (e.g., YOLO - You Only Look Once) This paradigm frames object detection as a single mathematical regression problem, evaluating the entire image simultaneously [9]. This approach sacrifices a marginal amount of precision for substantial computational speed, making it the theoretical standard for processing continuous, high-framerate video data from vehicle-mounted cameras [9].

1.4.3 Spatiotemporal Data Fusion

Detecting an object in a digital video frame is only useful for infrastructure management if that defect can be mapped to the physical real world. This requires *Spatiotemporal Data Fusion*, the theoretical process of combining data from multiple independent sensors (in this context, an optical camera and a GNSS/GPS receiver) into a single cohesive dataset.

1.5 Analysis of Road Infrastructure Monitoring

The inefficient monitoring of road surface damage presents a critical logistical and financial problem for municipalities. Traditional monitoring methods are currently divided into two extremes, both of which suffer from significant drawbacks.

1.5.1 Manual Reporting and Citizen Crowdsourcing

Most smaller municipalities rely on manual "road walks" or citizen reporting applications (e.g., *Waze* or local civic portals like *Odkaz pre starostu*). While inexpensive, these methods are inherently flawed. Citizen reporting is highly subjective and demographically biased towards major commuter routes, often ignoring peripheral neighborhoods. Consequently, it fails to provide an objective, real-time overview of the entire road network.

1.5.2 Specialized Profiler Vehicles (LiDAR)

At the other end of the technological spectrum are professional scanning systems utilized by national highway agencies. These specialized vehicles are equipped with laser

scanners (LiDAR) and provide millimeter precision, generating extensive 3D point clouds of the road surface [10].

However, the primary disadvantage of these systems is their substantial financial cost, often running into hundreds of thousands of euros for a single survey [10]. Because of this severe financial barrier, smaller districts can only afford these profiling services once every few years, leading to heavily outdated datasets that do not reflect rapid winter road degradation.

1.6 Comparative Analysis and Proposed Solution

Table 1.1 summarizes the critical differences between the current standard approaches and the novel solution proposed in this thesis, which utilizes the aforementioned YOLO26 architecture and telemetry fusion.

Evaluation Criteria	Manual / Crowdsourcing	Specialized Li-DAR Vehicles	Proposed Solution (YOLO26)
Implementation Cost	Very Low (Labor intensive)	Extremely High (Prohibitive)	Low (Action Camera + Consumer GPU)
Data Objectivity	Low (Subjective)	Very High (Precision)	High (Standardized model)
Update Frequency	Inconsistent	Annual / Bi-annual	Continuous / Daily via fleet
Spatial Precision	Approximate (Phone GPS)	Extremely High (RTK)	Hardware-dependent (Sync)
Output Format	Unstructured Photos/Text	Complex Point Clouds	Interactive Map & CSV file

Table 1.1: Comparison of road infrastructure monitoring methods and the proposed AI system.

The comparative analysis reveals a significant technological gap. Municipalities are

forced to choose between highly inaccurate subjective data and financially inaccessible laser data. The practical implementation detailed in the subsequent chapters bridges this exact gap by applying single-stage object detection to standard video footage, providing local governments with a high-frequency, objective, and affordable mapping tool.

Chapter 2

System Specification and Objectives

Following the theoretical analysis of artificial intelligence in public administration and the identification of the technological gap in current road monitoring methodologies, this chapter defines the precise objectives and specifications for the practical implementation of this thesis. It outlines the functional and non-functional requirements that the proposed pothole detection system must fulfill to be considered a viable tool for municipal deployment.

2.1 Main Objectives

The primary objective of this practical implementation is to design, train, and develop an automated, cost-effective software system capable of detecting road potholes from standard video recordings and accurately localizing them on a geographical map.

The system is designed to act as an asynchronous assistance tool for local municipalities. Instead of relying on expensive specialized LiDAR vehicles or subjective citizen reports, the proposed solution utilizes standard, off-the-shelf action cameras (such as the GoPro Hero series). These cameras can be easily mounted on the dashboards or windshields of existing municipal vehicle fleets, such as garbage trucks, public transport buses, or even bicycles. By processing the recorded data offline, the system must independently identify road damage, filter out visual anomalies, and generate a comprehensive spatial output.

2.2 Functional Requirements

To ensure the system is viable for real-world municipal application, it must satisfy several key functional requirements. These requirements define the specific behaviors, logical steps, and tasks the software must successfully accomplish.

- **Data Ingestion:** The system must be capable of processing standard, high-definition video formats (e.g., MP4) that contain embedded telemetry metadata, specifically GPS coordinates recorded concurrently with the video stream.
- **Automated Object Detection:** The system must analyze the video feed sequentially, frame by frame, and detect potholes with a high degree of confidence. This detection must be powered by a custom-trained Convolutional Neural Network (CNN).
- **False Positive Mitigation:** A critical requirement of the system is robustness against visual anomalies. The software must be explicitly trained to distinguish between actual asphalt degradation (potholes) and similar circular road features, specifically metallic manhole covers and sewer grates.
- **Telemetry Extraction:** The system must systematically extract the embedded GPS coordinates logged by the camera's internal receiver and structure them into a readable data format alongside their precise timestamps.
- **Data Fusion and Synchronization:** The software must accurately pair the visual detections with the extracted spatial data. When the AI model identifies a pothole within a video frame, the system must extract that frame's exact timestamp, query the generated telemetry log, and assign the closest corresponding GPS coordinate to the detected defect.
- **Optimize Spatial Mapping through Peak Confidence Deduplication:** To prevent the final geographical map from being cluttered with redundant markers caused by the vehicle passing a single defect over multiple frames, the system evaluates the AI confidence scores across consecutive detections, isolates the single frame with the highest confidence, and uses that frame's coordinate as the definitive location of the pothole.
- **Geospatial Output Generation:** The final output of the system must be a standardized geospatial format, specifically an interactive geographical map and a comprehensive CSV file that visualizes the pinpointed locations of the road damage, allowing road administrators to navigate to the defect easily.

2.3 Non-Functional Requirements

In addition to the functional capabilities, the system must adhere to specific constraints regarding its performance, execution environment, and scalability.

- **Hardware Efficiency and Memory Management:** The inference process (the application of the trained AI model to the high-resolution video) must be optimized to run on consumer-grade to high-end desktop hardware (e.g., an NVIDIA RTX GPU). The system must manage Video RAM (VRAM) and system paging files effectively to prevent memory overflow crashes during the processing of large video files.
- **Cost-Effectiveness:** The hardware required for data collection must remain strictly off-the-shelf and affordable, ensuring that the financial barrier to entry for smaller municipalities remains as low as possible.
- **Modularity:** The Python scripts handling the AI inference and the GPS extraction should be modular. This ensures that future developers can easily upgrade the underlying YOLO architecture to a newer or lighter version without needing to rewrite the entire telemetry and mapping pipeline.

2.4 Target Audience and Use Cases

The final software artifact is tailored specifically for municipal road maintenance departments. By providing both a visual map and raw CSV data, the system allows road administrators to transition from a reactive maintenance strategy (waiting for citizen complaints) to a proactive one. Maintenance crews can use the generated outputs to optimize their repair routes, assess the severity of road degradation across different city districts, and allocate public funds more efficiently based on objective AI-generated data.

To illustrate the intended functional workflow of the system, Figure 2.1 outlines the interaction between human operators and the automated processing pipeline.

As depicted, field workers simply record the data during their standard routes and hand off the raw MP4 files. The IT administrator then feeds this data into the two primary backend modules, the telemetry extractor and the YOLO neural network. Finally, the data fusion script mathematically synchronizes the outputs, delivering actionable maintenance data without requiring manual visual inspection of the footage. Ultimately, this workflow bridges the gap between raw field recordings and strategic municipal planning.

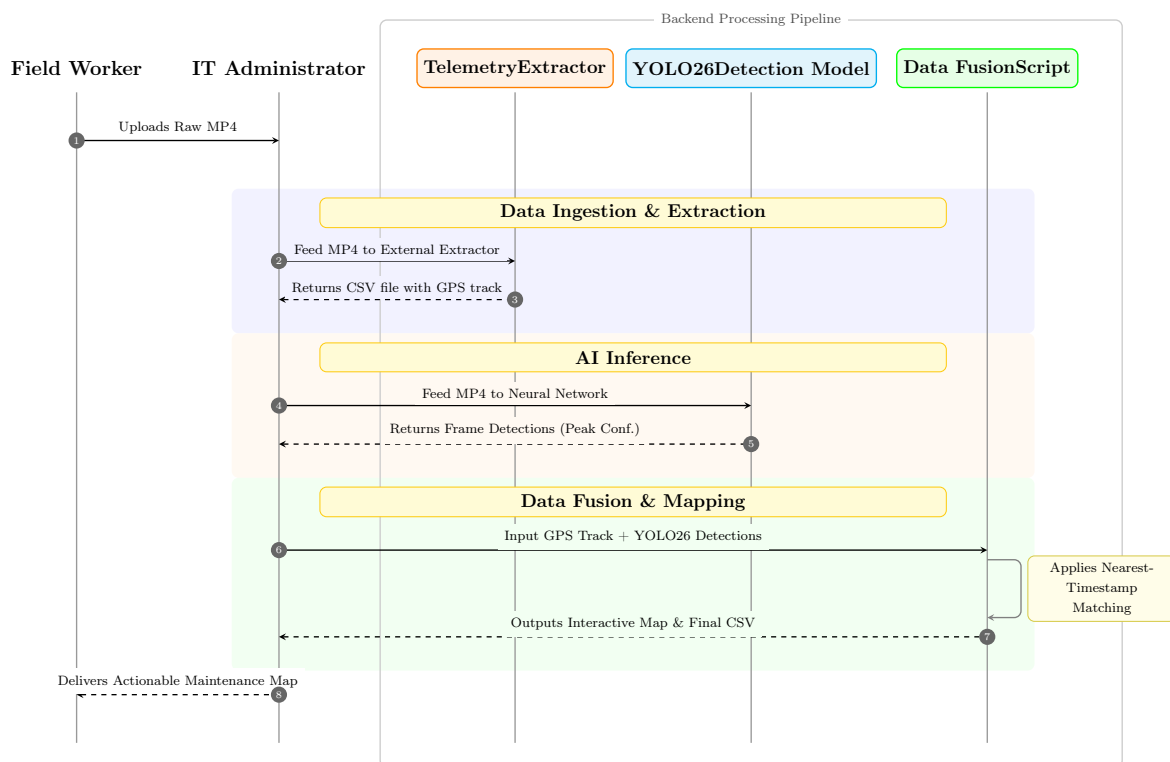


Figure 2.1: UML Sequence Diagram illustrating the intended operational workflow.

Chapter 3

System Architecture and Design

Following the theoretical framework and system specifications established in the previous chapters, this section translates those requirements into a concrete technical architecture. It details the logical design of the proposed solution, the structural flow of data from physical acquisition to final visualization, and the conceptual rules governing the artificial intelligence model and the spatiotemporal fusion algorithm.

3.1 Data Acquisition and Recording

The foundation of the proposed architecture relies on passive data collection. Rather than employing specialized surveying vehicles, the system is designed to utilize standard, consumer-grade action cameras (such as the GoPro Hero series).

These cameras are mounted on the dashboards or windshields of existing municipal vehicles. As the vehicle performs its standard daily route (e.g., waste collection or public transport), the camera records high-definition video of the road surface. Crucially, these modern action cameras simultaneously record internal, high-frequency telemetry data, embedding the precise GPS coordinates and timestamps directly into the raw MP4 video file as it is captured.

3.2 Telemetry Extraction Strategy

Once the physical data is recorded, the raw MP4 file must be separated into two distinct data streams: the visual video feed and the spatial geographic data.

Because the telemetry is embedded within the video’s metadata structure, the architecture utilizes a dedicated third-party extraction tool. This external software parses the video file, isolates the embedded GPS metadata, and exports it into a structured, readable log (typically a CSV file). This extracted log acts as the spatial database for the route, containing a continuous sequence of precise GPS coordinates accurately

paired with their corresponding temporal timestamps. The complete structural flow of this data separation, visual inference, and subsequent fusion pipeline is illustrated in Figure 3.1.

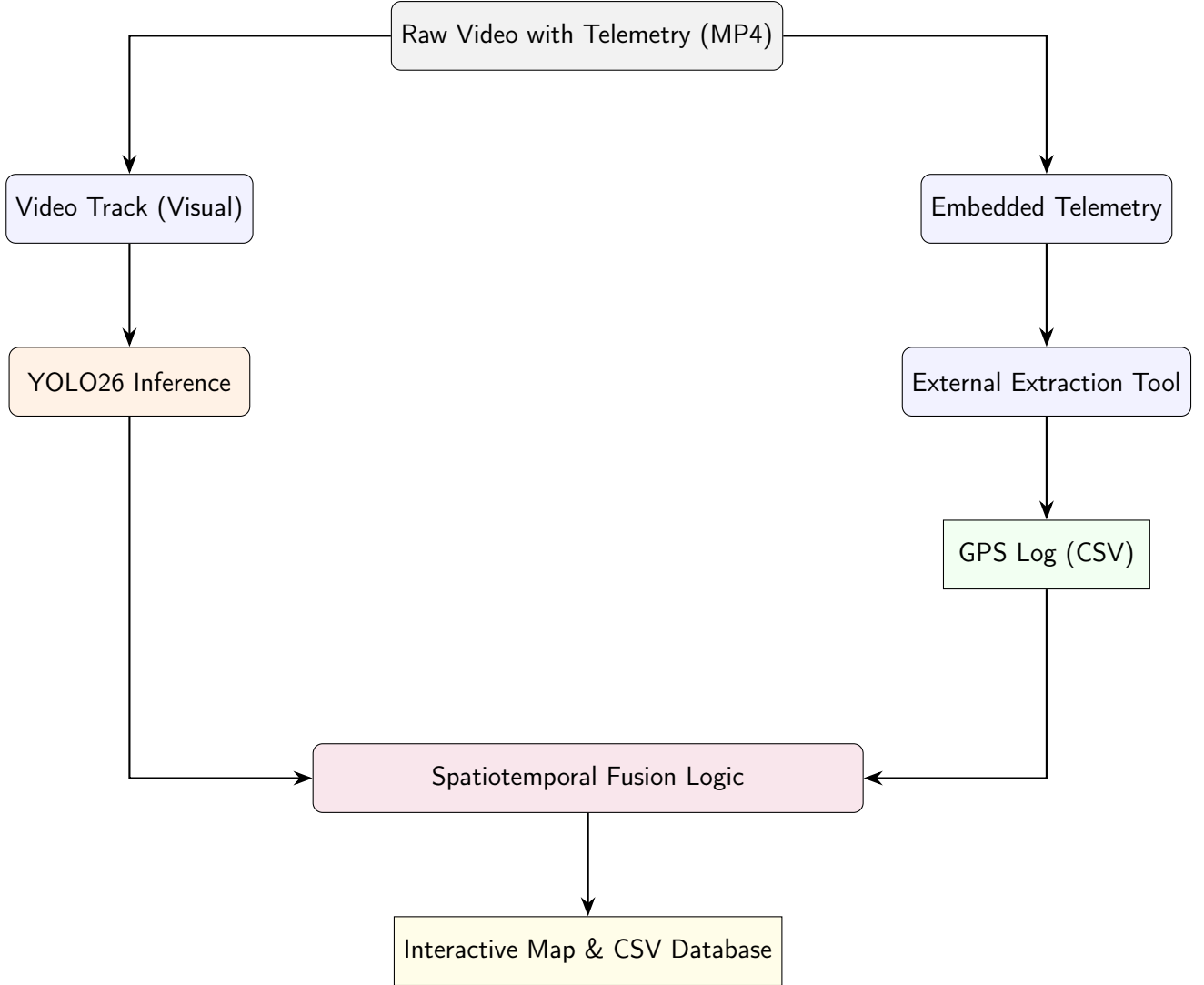


Figure 3.1: High-level architectural flowchart illustrating the data separation, inference, and fusion pipeline.

3.3 Dataset Construction and Model Training

The visual detection capabilities of the system are entirely dependent on the quality of the data used to train the artificial intelligence. Because road degradation topologies vary significantly based on regional climate and asphalt composition, a hybrid dataset strategy was employed to train the YOLO26 detection model.

3.3.1 Hybrid Dataset Aggregation

To establish a robust baseline for pothole recognition, the system utilizes a pre-existing, open-source dataset sourced from the Roboflow platform, consisting of 1,243 annotated images. However, to guarantee that the model performs accurately on local infrastructure, this baseline was augmented. Custom footage was recorded locally, sampled into discrete frames, and manually annotated, contributing an additional 126 images. The model was then explicitly trained on this merged dataset of 1,369 total images, ensuring it could generalize effectively while maintaining high accuracy in its specific target deployment environment.

3.3.2 False Positive Mitigation

To structurally prevent the AI from misclassifying circular metallic features, such as manhole covers and sewer grates as asphalt degradation, the training dataset incorporated "Background Images." These are negative examples containing common road infrastructure but entirely lacking bounding box annotations. By training the model on these empty frames, the training process guided the neural network to learn the visual distinction between an actual road cavity and flat, non-hazardous infrastructure.

3.4 Object Tracking and Deduplication Logic

With the model trained and the GPS log extracted, the system initiates the core visual processing phase. The video stream is evaluated sequentially, frame-by-frame, by the AI. However, simply detecting a defect in an isolated frame is insufficient for a continuous video feed.

3.4.1 Sequential Object Tracking

As a vehicle approaches a physical defect, that specific pothole will appear across multiple consecutive video frames. The system's algorithm is designed to assign a unique tracking identifier to a pothole upon its first detection. It then maintains and updates that identifier as the object moves through the camera's field of view over time. To filter out random visual glitches or temporary reflections, the algorithm enforces a minimum lifespan rule: a defect must be consistently tracked across a predefined number of consecutive frames before it is officially recognized by the system.

3.4.2 Peak Confidence Deduplication

Logging every single frame in which a pothole appears would generate substantial data redundancy and visually clutter the final map. To solve this, the algorithm utilizes a "Peak Confidence" deduplication strategy.

As the algorithm tracks a specific pothole across a sequence of frames, it continuously evaluates the neural network's predictive confidence score. The system logically isolates the single optimal frame where the AI's detection confidence reaches its maximum. The algorithm extracts and safely stores the exact timestamp of this peak-confidence frame, subsequently discarding all other lower-confidence frames of that same pothole from the sequence.

3.5 Spatiotemporal Synchronization

Once the deduplication process is complete, the system possesses a clean list of highly confident pothole detections, each associated with a single, precise video timestamp. The final architectural challenge is mapping these visual timestamps to the physical world.

The system accesses the structured GPS log generated earlier by the third-party extractor. Because the video frame rate (e.g., 30 FPS) operates at a different frequency than the internal GPS receiver, direct one-to-one matching is impossible. As visually demonstrated in Figure 3.2, the architecture solves this via a nearest-timestamp mathematical approach. The algorithm calculates the absolute temporal difference between the saved timestamp of the peak-confidence video frame and the high-frequency timestamps available in the extracted GPS log. By identifying the minimum absolute difference, the algorithm accurately assigns the closest recorded geographic coordinate to the defect, successfully anchoring the digital detection to the physical trajectory of the vehicle.

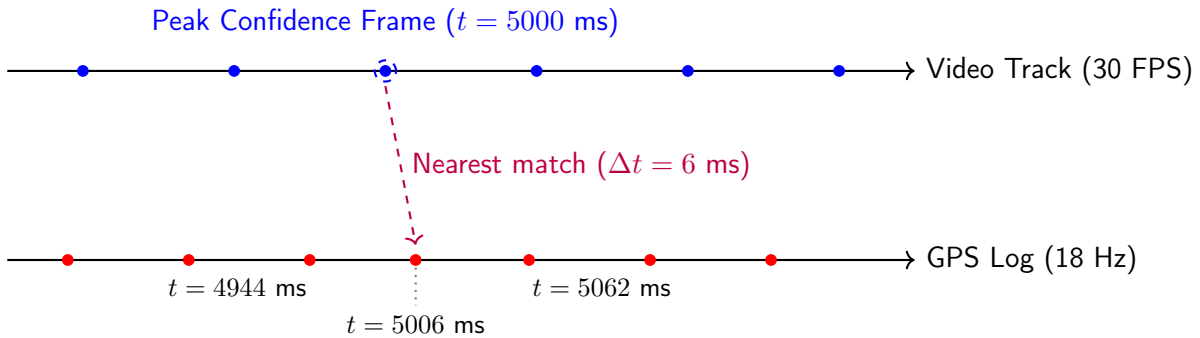


Figure 3.2: Visual representation of the spatiotemporal fusion logic, synchronizing a discrete video frame timestamp to the nearest continuous high-frequency GPS coordinate.

3.6 System Output Generation

The final stage of the architectural design translates the synchronized data into actionable administrative tools. The system is designed to automatically generate a three-tiered output structure:

- **Interactive Geospatial Map:** An interactive, browser-based map rendering each pinpointed defect as a distinct geographical marker. This interface allows road maintenance crews to immediately assess the spatial distribution of road damage across a district.
- **Structured CSV Database:** A raw, standardized data file containing the unique identifier, the source video name, the peak AI confidence score, and the precise latitude and longitude for every detected defect.
- **Annotated Evidence Imagery:** As illustrated in Figure 3.3, for every defect logged in the geospatial database, the system automatically saves a local copy of the exact peak-confidence video frame. This image is overlaid with a standardized bounding box and confidence score generated by the YOLO26 model, accurately isolating the pothole. This provides administrators with immediate visual proof of the defect's severity before dispatching physical repair crews.



Figure 3.3: Example of annotated evidence: The YOLO26 model successfully localizing a road defect at its peak confidence frame.

Chapter 4

Software Implementation and Development

With the architectural framework and system requirements clearly defined, this chapter details the practical software development phase. It outlines the preparation of the visual data, the specific hyperparameter configurations utilized during the artificial intelligence training phase, and the programmatic logic written to construct the final spatiotemporal fusion pipeline.

4.1 Data Preprocessing and Environment Setup

The core processing pipeline was developed in the Python programming language, utilizing consumer-grade GPU hardware to accelerate neural network inference. However, before the model training and inference scripts could be executed, several significant data preprocessing hurdles had to be resolved.

First, the initial dataset exported from the Roboflow platform lacked a predefined structural split, despite the dataset being split on the Roboflow platform itself. All annotated images were initially grouped into a single repository, requiring manual intervention to prepare the data for the network architecture.

Second, a decoding bottleneck occurred regarding the raw video files during the inference phase. The standard Python OpenCV libraries struggled to decode raw GoPro `.mp4` containers because they are heavily multiplexed with proprietary, high-frequency metadata tracks. Attempting to read these raw files directly via `cv2.VideoCapture` resulted in decoding errors and corrupted frame extraction. To solve this, the raw footage was routed through a third-party processing program to safely re-encode the visual stream into a standard, metadata-free MP4 file, ensuring 100% compatibility with the OpenCV pipeline.

4.2 Model Training and Hyperparameter Configuration

The visual detection capabilities of the system are powered by the state-of-the-art YOLO26 (You Only Look Once) architecture, released by Ultralytics in early 2026. This specific model iteration was selected due to its native end-to-end NMS-free (Non-Maximum Suppression) inference and removal of the DFL module, which significantly accelerates inference processing on consumer hardware and edge devices [11]. Training a custom YOLO26 model requires strict configuration of both the underlying dataset formatting and the specific hyperparameters, as these directly dictate the network’s ability to learn and generalize.

4.2.1 Data Leakage Anomaly and Retraining Strategy

Prior to the successful deployment of the final model, an initial training phase was conducted which resulted in a critical methodological anomaly. During this first iteration, the model rapidly achieved an unusually high evaluation metric of approximately 95%. However, subsequent field testing revealed that the network struggled significantly to detect potholes in novel, unseen video footage.

An investigation into the dataset configuration file (`data.yaml`) revealed a severe structural flaw: the directory paths defining the training, validation, and testing subsets were inadvertently mapped to the exact same root folder. Consequently, the neural network suffered from complete *data leakage*. The model was validating its performance on the exact same images it had already processed during the training epochs, allowing it to simply memorize the data rather than learn the generalized topological features of asphalt degradation.

To rectify this programmatic error, the initial model weights were entirely discarded. The raw dataset, comprising 1,369 total images (1,243 from the original open-source dataset and 126 manually added images), was strictly partitioned into mutually exclusive directories. Utilizing a 75/15/10 split ratio, the data was divided into 1,027 images for training, 205 for validation, and 137 for testing. The neural network was then retrained completely from scratch to establish a mathematically sound baseline. Following this correction, the retrained model achieved a mean Average Precision (mAP@0.5) of 0.841 and a peak F1-score of 0.79. While numerically lower than the overfitted 95% metric of the flawed initial model, these evaluation metrics accurately reflect the network’s true capability to generalize to novel road environments.

4.2.2 Hyperparameter Tuning

Once the dataset integrity was verified, the final training process was initialized with several critical parameters configured to optimize the detection of small road defects:

- **Image Size (imgsz):** By default, YOLO26 compresses training images to 640x640 pixels. Because potholes often appear as small clusters of pixels in the distance, the `imgsz` parameter was explicitly increased to 1280. Higher input resolutions drastically improve small-object detection accuracy, although they require exponentially more Video RAM (VRAM) during the backpropagation phase.
- **Batch Size (batch):** This defines how many images the neural network processes simultaneously before updating its internal weights. This was calibrated based on the available GPU VRAM to ensure stable training without memory overflow crashes.
- **Epochs:** This parameter defines the total number of complete passes the algorithm makes through the entire training dataset. The model was trained sufficiently long to allow the validation loss metrics to plateau, ensuring the network had fully converged.

4.2.3 False Positive Mitigation via Background Images

During the validation phases of the retrained model, a secondary challenge was identified: the network consistently suffered from False Positives. The AI frequently misclassified circular, metallic sewer grates and manhole covers as dangerous asphalt potholes.

Because a neural network only knows what it is explicitly taught, it had learned to associate "dark circular shapes on the road" with the "pothole" class. To rectify this, the custom dataset of 126 locally sourced images was strategically curated to include 50 "Background Images." These 50 negative examples consisted of image frames featuring manhole covers, shadows, and sewer grates, which were submitted to the trainer with completely blank annotation files. By constraining the algorithm to evaluate these negative samples during training, the network mathematically learned the subtle textural differences between a physical cavity in the asphalt and a flat metallic cover, significantly reducing the occurrence of false positives.

4.3 Inference and Sequential Object Tracking

With the model successfully trained and compiled into a `best.pt` weights file, the system initiates the inference script. To streamline the operational workflow for municipal administrators, the inference script incorporates automated batch processing.

Rather than requiring a human operator to manually execute the program for every single video, the script is designed to automatically scan a designated input directory upon launch. It utilizes string-matching logic to pair video files (.mp4) with their independently extracted telemetry logs (.csv) based on their identical base filenames.

Once a valid pair is established by the script, the system initiates the model inference. The core visual processing occurs within a continuous `while` loop that reads the sanitized video frame-by-frame.

```

1 results = model.track(frame, persist=True, imgsz=1280, conf=0.55,
    verbose=False)
2 if results[0].boxes.id is not None:
3     ids = results[0].boxes.id.cpu().numpy()
4     boxes = results[0].boxes.xyxy.cpu().numpy()
5     confidences = results[0].boxes.conf.cpu().numpy()

```

Algorithm 4.1: Execution of the YOLO26 tracking algorithm and parameter extraction.

Setting the `persist=True` parameter instructs the Ultralytics tracker to retain object identities across consecutive frames. To prevent map clutter, a localized dictionary named `active_objects` acts as a deduplication filter.

```

1 if tid not in active_objects:
2     active_objects[tid] = {"start": frame_number, "last": frame_number,
    , "cts": current_cts, "max_c": c, "box": b, "img": frame.copy()}
3 else:
4     active_objects[tid]["last"] = frame_number
5     if c > active_objects[tid]["max_c"]:
6         active_objects[tid].update({"max_c": c, "box": b, "img": frame
    .copy()})

```

Algorithm 4.2: Deduplication logic isolating the maximum confidence score.

This logic loop ensures that only the frame with the absolute highest confidence score (`max_c`) is retained in the system memory. Lower-confidence frames of the exact same defect are continuously overwritten.

4.4 Spatiotemporal Synchronization Algorithm

Once a defect falls outside the camera's field of view, the digital timestamp of its peak-confidence frame is mapped to the physical world. Because the extracted telemetry records at approximately 18 Hz, the script must perform a nearest-neighbor mathematical search.

```

1 idx = (gps_df['cts'] - data["cts"]).abs().idxmin()
2 row = gps_df.iloc[idx]
3 lat, lon = row['GPS (Lat.) [deg]'], row['GPS (Long.) [deg]']

```

Algorithm 4.3: Nearest-timestamp matching utilizing Pandas vectorization.

By utilizing the Pandas `abs().idxmin()` function, the script subtracts the video timestamp from the entire telemetry column simultaneously. It mathematically isolates the exact row index (`idx`) with the minimum temporal difference, snapping the visual detection to the physical GPS track.

4.5 Geospatial Output and Visualization

The final programming task involves packaging the synchronized data into actionable outputs. First, OpenCV draws a standardized red rectangle and writes the exact confidence percentage directly onto the duplicated image frame, generating the Annotated Evidence Imagery.

Next, the system utilizes the Folium library to generate an interactive HTML map. During initial development, the default OpenStreetMap (OSM) rendering tiles were utilized. However, an issue emerged: the default OSM tile servers flagged the automated rendering requests. Because the Python script was executing calls without passing a verified browser **User-Agent** header or dedicated API authentication, the OSM servers embedded prominent "Access blocked" text overlays directly across the rendered map tiles. While the geospatial data generated correctly, these obstructive watermarks rendered the output visually cluttered and entirely unsuitable for professional municipal use.

To resolve this access limitation and ensure stable map generation, the Folium initialization parameters were modified to call an alternative, restriction-free tile provider.

```

1 m = folium.Map(location=[all_potholes[0]["Lat"], all_potholes[0]["Lon"]],
2                 zoom_start=14, tiles='CartoDB positron')
3 for v in all_potholes:
4     popup_text = f"Pothole #{v['ID']}<br>Video: {v['Video']}<br>
5     Confidence: {v['Conf']*100:.1f}%"
6     folium.Marker([v["Lat"], v["Lon"]], popup=popup_text, icon=folium.
7     Icon(color="red")).add_to(m)
8 m.save(os.path.join(output_folder, "Output_Map.html"))

```

Algorithm 4.4: Generation of the interactive Folium map utilizing alternative tiles.

By explicitly defining `tiles='CartoDB positron'`, the system bypasses the verification block and successfully forces Folium to render a light-gray, high-contrast basemap.

Chapter 5

Evaluation and Results

This chapter details the quantitative evaluation of the final trained YOLO26 detection model. Following the resolution of the data leakage anomaly discussed in Chapter 4, the model was evaluated against the partitioned testing dataset (representing 10% of the total data, or 137 novel images). This ensures that the following metrics accurately reflect the system’s real-world generalization capabilities on previously unseen road infrastructure.

5.1 Evaluation Metrics

To assess the performance of the object detection algorithm, standard computer vision metrics were utilized. The evaluation relies on the calculation of True Positives (TP - correctly identified potholes), False Positives (FP - background anomalies incorrectly identified as potholes), and False Negatives (FN - actual potholes missed by the model). From these base values, the following primary metrics are derived:

- **Precision:** The ratio of correctly predicted positive observations to the total predicted positive observations. It answers the question: *Of all the objects the AI labeled as potholes, how many were actually potholes?*
- **Recall:** The ratio of correctly predicted positive observations to all actual observations in the specific class. It answers the question: *Of all the physical potholes present in the dataset, what fraction did the AI successfully find?*
- **F1-Score:** The harmonic mean of Precision and Recall, providing a single metric that balances the trade-off between the two.
- **Mean Average Precision (mAP@0.5):** The definitive standard for evaluating object detection models, calculating the average precision across all recall values at an Intersection over Union (IoU) threshold of 0.50.

5.2 Confidence Threshold Analysis

In automated object detection, the system calculates a confidence probability (ranging from 0.0 to 1.0) for every bounding box it generates. Adjusting the minimum confidence threshold required to accept a detection directly influences the balance between Precision and Recall.

As illustrated in Figure 5.1, the model’s F1-Score was evaluated across the entire spectrum of confidence thresholds. The peak F1-Score achieved by the model was 0.79 at a confidence threshold of 0.278.

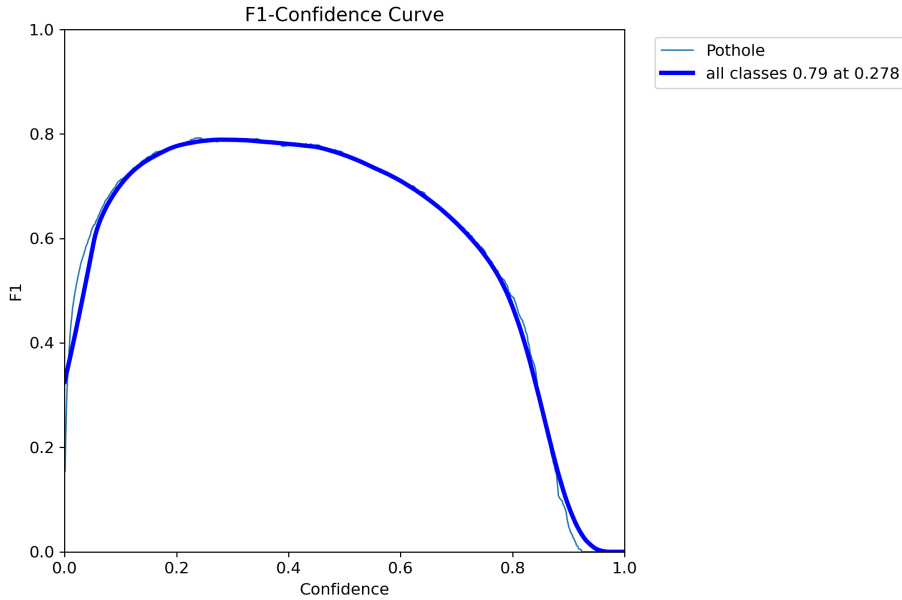


Figure 5.1: F1-Confidence Curve demonstrating the optimal balance between precision and recall at a threshold of 0.278.

This optimal threshold dictates the operational deployment parameters. Setting the threshold too low (e.g., < 0.10) maximizes Recall, as seen in Figure 5.3, meaning the system identifies almost every physical defect, but at the cost of being overwhelmed by False Positives. Conversely, setting the threshold too high (e.g., > 0.80) maximizes Precision, as seen in Figure 5.2, ensuring that any detection is virtually guaranteed to be a pothole, but resulting in a severe drop in Recall, causing the system to ignore dozens of legitimate defects. The peak F1-Score of 0.79 represents a highly stable mathematical optimum for balancing municipal reporting accuracy without overburdening administrators with false data.

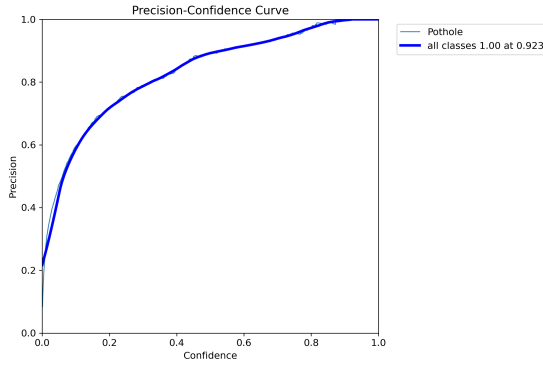


Figure 5.2: Precision-Confidence Curve.

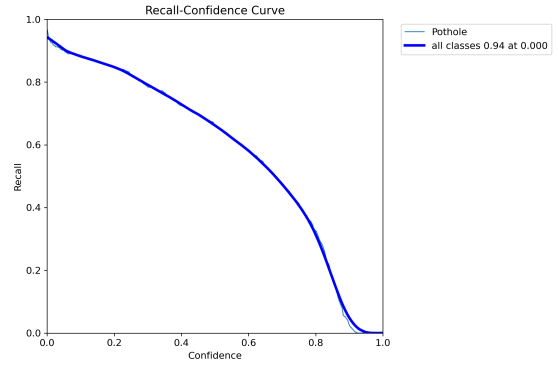


Figure 5.3: Recall-Confidence Curve.

5.3 Overall Performance and Precision-Recall Dynamics

The comprehensive performance of the model is best visualized through the Precision-Recall (PR) Curve, shown in Figure 5.4. The area under this curve determines the mAP@0.5 score.

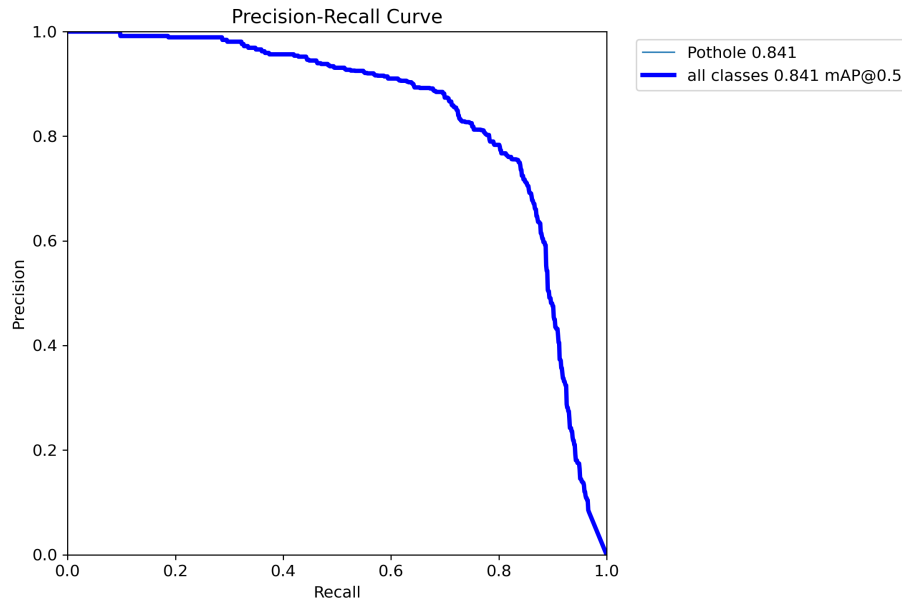


Figure 5.4: Precision-Recall Curve illustrating the model's high overall detection capability.

The trained YOLO26 model achieved an mAP@0.5 of 0.841. Given that asphalt degradation lacks a standardized geometric shape, color, or depth, detecting these defects in unstructured road environments presents a complex computer vision challenge. The model's performance on unseen data indicates that the algorithm has effectively generalized the varying topographical features of road damage, establishing a viable

baseline for automated detection.

5.4 Classification Accuracy and Error Analysis

To gain deeper insight into the specific performance and failure modes of the model, a Confusion Matrix was generated against the testing subset.

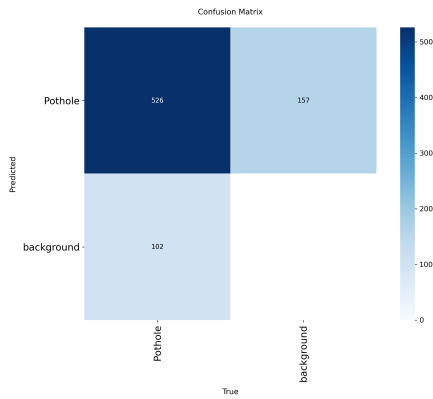


Figure 5.5: Absolute Confusion Matrix.

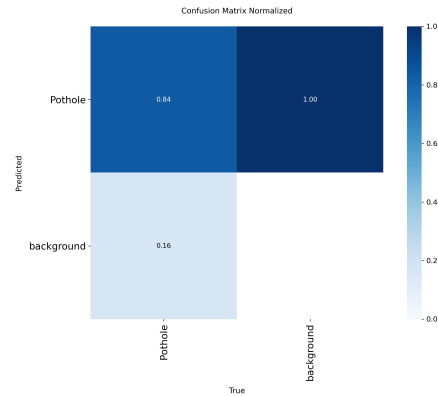


Figure 5.6: Normalized Confusion Matrix.

As depicted in the normalized matrix (Figure 5.6), the model achieved a True Positive rate of 84% (0.84). This indicates that the majority—84% of all physical potholes present in the unseen testing footage—were successfully localized and classified by the YOLO26 architecture.

The matrix also reveals that the system exhibits a False Negative rate of only 16% (0.16). In 102 instances (Figure 5.5), the algorithm evaluated an actual pothole but classified it as the background road surface. This phenomenon is largely attributable to severe edge cases in the input data, such as extremely shallow pavement depressions, defects entirely obscured by vehicular shadows, or potholes captured in heavy motion blur during high-speed transit.

Regarding False Positives, the matrix records 157 instances where background anomalies were incorrectly predicted as potholes. While the introduction of "Background Images" (discussed in Section 4.2.3) minimized misclassifications of standardized infrastructure like manhole covers, natural asphalt anomalies—such as complex shadow intersections or freshly patched asphalt—can still occasionally trigger a positive detection. However, given the scale of the processed data, a False Positive count of 157 is operationally acceptable for a passive monitoring tool.

5.5 Hardware Environment and Training Dynamics

To prove the accessibility of the proposed system for municipal deployment, the computational training pipeline was executed on a high-end consumer workstation rather than a specialized enterprise server. The environment was powered by an NVIDIA GeForce RTX 3090 GPU, an AMD Ryzen 9 9950X3D processor, and 32GB of system RAM.

The 24GB VRAM capacity of the RTX 3090 was necessary to accommodate the massive tensor matrices generated by the 1280x1280 input resolution (`imgsz=1280`) during the backpropagation phase without encountering memory overflow errors (The system used about 18GB VRAM during the training phase).

The neural network training was conducted in a two-phase approach on the 1,369-image dataset. The initial phase consisted of training the model completely from scratch for a full 200 epochs. To further optimize the network weights, a second continuous training phase was initiated with a target of an additional 200 epochs. However, this second phase was automatically halted by the algorithm after approximately 100 epochs when the validation loss metrics plateaued, preventing the model from overfitting. Throughout the entire process, the computational pipeline maintained a stable iteration speed, with each individual epoch taking approximately 50 to 51 seconds to complete, thus taking about 4 and a half hours to complete the entire training process.

5.6 Real-World Pipeline Evaluation

While standard computer vision metrics validate the model’s recognition capabilities on static images, the ultimate test of the system requires evaluating the entire spatiotemporal pipeline in a continuous, dynamic environment.

To conduct this qualitative evaluation, a real-world video recording (`test.mp4`) and its corresponding telemetry log (`test.csv`) were placed into the system’s input directory. The test footage covered a continuous drive and, based on a manual review of the road surface, contained exactly 15 distinct, physical potholes.

Demonstrating its batch-processing capabilities, the automated Python pipeline scanned the input folder, successfully identified and paired the associated files based on string-matching, and initiated the processing sequence entirely without human intervention. Along the evaluated route, the YOLO26 tracking and deduplication logic generated a total of 16 defect markers. Upon cross-referencing these outputs with the manual evaluation, 10 of these markers correctly identified and mapped the target physical potholes.

During the error analysis of this specific route, the system recorded 5 missed defects (False Negatives) and generated 6 false positive markers. Importantly, this eval-

uation highlighted a fundamental challenge in automated infrastructure mapping: the subjective definition of a road defect. Even during the manual verification process, determining the exact threshold where a minor asphalt depression, a sealed crack, or a rough patch formally qualifies as a "pothole" proved inherently ambiguous. Consequently, the model's false positives and false negatives predominantly occurred in these subjective gray areas.



Figure 5.7: Example of a False Positive: The system incorrectly identifying a non-critical surface anomaly (e.g., shadow or patch) as a pothole.



Figure 5.8: Example of a False Negative: The system failing to detect a physical pothole, likely due to motion blur.

To mitigate these discrepancies and improve the operational reliability of the software in the future, two distinct advancements are necessary. First, the neural network requires further dataset refinement, incorporating thousands of additional, highly varied examples of edge-case road topography. Second, successful municipal deployment will require the end-user (i.e., the road maintenance department) to establish a strict, standardized physical definition of what constitutes a pothole, ensuring the AI can be trained and evaluated against unambiguous criteria rather than subjective human interpretation.

The system's operational viability can be quantified using standard metrics. Based on the manual review of the test route, the end-to-end spatiotemporal pipeline achieved a Precision of 62.5% (10 correct detections out of 16 total generated markers) and a Recall of 66.6% (10 successfully mapped potholes out of 15 actual physical defects). This yields a real-world pipeline F1-score of 0.645. Although the real-world pipeline F1-score of 0.645 is lower than the 0.79 achieved during controlled static testing, the results validate the temporal matching algorithm's underlying logic and establish a baseline for automating the extraction of geospatial data from continuous video feeds.

5.7 Summary of Results

To provide a clear overview of the system’s capabilities, all primary data points and evaluation metrics discussed in this chapter are aggregated in Table 5.1.

Table 5.1: Comprehensive Summary of System Performance and Evaluation Metrics

Metric / Parameter	Achieved Value
Hardware & Training Parameters	
Compute Hardware	NVIDIA RTX 3090
Dataset Size (Total / Test Split)	1,369 images / 137 images (10%)
Total Training Epochs	~300
Average Training Time per Epoch	50 - 51 seconds
Phase 1: Static Image Evaluation (Testing Dataset)	
Mean Average Precision (mAP@0.5)	0.841
Optimal Confidence Threshold	0.278
Peak F1-Score	0.79
True Positive Rate	84%
False Negative Rate	16%
Absolute False Positives	157
Absolute False Negatives	102
Phase 2: Real-World Pipeline (Continuous Video test.mp4)	
Total Physical Potholes	15
Total System Detections	16
True Positives	10
False Negatives	5
False Positives	6
Pipeline Precision	62.5% (10/16)
Pipeline Recall	66.6% (10/15)
Pipeline F1-Score	0.645

Conclusion

The primary objective of this bachelor’s thesis was to design, train, and implement an automated software system capable of detecting and mapping road surface degradation using artificial intelligence. By leveraging the YOLO26 object detection architecture and custom spatiotemporal data fusion, the project sought to bridge the technological gap between subjective manual citizen reporting and expensive, specialized LiDAR surveys.

The quantitative and qualitative evaluations detailed in this work confirm that the main objectives were met. The custom-trained neural network, built upon a curated dataset of 1,369 images, reached a mean Average Precision (mAP@0.5) of 0.841 and a peak F1-score of 0.79. Furthermore, the system’s practical application was demonstrated through real-world testing. The automated Python pipeline processed continuous video footage alongside extracted GPS telemetry, applied peak-confidence deduplication logic to filter out redundant detections, and autonomously generated interactive geospatial maps. This serves as a functional proof of concept, indicating that standard action cameras mounted on municipal fleets can serve as a viable passive scanning network for road maintenance.

Despite these results, the development process highlighted several key areas for future improvement, both in terms of machine learning accuracy and software usability, which must be addressed prior to municipal deployment.

First, while the model achieved an 84% true positive rate on the testing data, the robustness of the artificial intelligence requires continual improvement through iterative dataset expansion. The error analysis of real-world video footage highlighted a fundamental challenge in automated infrastructure mapping: the subjective definition of road defects. Ambiguous surface anomalies, such as shallow depressions, sealed cracks, or complex shadows, constituted the majority of the remaining false positives and false negatives. Resolving this requires two steps: exposing the neural network to a larger, geographically diverse dataset across various weather and lighting conditions, and establishing a strict, standardized physical definition of what constitutes an actionable pothole to separate subjective human interpretation from the evaluation process.

Second, the overall usability of the software artifact must be refined for non-technical end-users. Currently, the system operates as a collection of backend Python

scripts requiring a designated input directory and an initialized programming environment. Future development should focus on transitioning this code-base into a compiled, standalone desktop or web application featuring a Graphical User Interface (GUI).

Finally, the current data preprocessing workflow must be natively integrated into the proposed application. The present pipeline requires road administrators to utilize third-party tools to extract the embedded GPS telemetry and re-encode the raw, multiplexed MP4 files to ensure compatibility with OpenCV. Incorporating native video decoding and telemetry parsing directly into the application's source code would eliminate these external dependencies. This integration would allow users to input raw camera files into the application and generate a finished interactive map in a single automated process.

Ultimately, this thesis demonstrates the functional feasibility of integrating smart city technologies into local infrastructure management. By transitioning theoretical machine learning models into applied software engineering tools, municipalities can improve resource allocation, reduce manual monitoring, and support a transition toward a proactive, data-driven model of road maintenance.

Bibliography

- [1] Vincent J Straub, Deborah Morgan, Jonathan Bright, and Helen Margetts. Artificial intelligence in government: Concepts, standards, and a unified framework. *Government Information Quarterly*, 40(4):101881, 2023.
- [2] Colin Van Noordt and Luca Tangi. The dynamics of ai capability and its influence on public value creation of ai within public administration. *Government Information Quarterly*, 40(4):101860, 2023.
- [3] Khalifa Alhosani and Saadat M Alhashmi. Opportunities, challenges, and benefits of ai innovation in government services: a review. *Discover Artificial Intelligence*, 4(1):18, 2024.
- [4] Illugi Torfason Hjaltalin and Hallur Thor Sigurdarson. The strategic use of ai in the public sector: A public values analysis of national ai strategies. *Government Information Quarterly*, 41(1):101914, 2024.
- [5] City of Amsterdam. Object detection kit amsterdam - automatic detection of garbage. Technical report, Interoperable Europe Portal, European Commission, 2020.
- [6] Hamsa Bastani, Kimon Drakopoulos, Vishal Gupta, Ioannis Vlachogiannis, Christos Hadjichristodoulou, Pagona Lagiou, Gkikas Magiorkinis, Dimitrios Paraskevis, and Sotirios Tsiodras. Efficient and targeted covid-19 border testing via reinforcement learning. *Nature*, 599(7883):108–113, 2021.
- [7] Tan Yigitcanlar, Rita Yi Man Li, Prithvi Bhat Beeramoole, and Alexander Paz. Artificial intelligence in local government services: Public perceptions from australia and hong kong. *Government Information Quarterly*, 40(3):101833, 2023.
- [8] Yann LeCun, Yoshua Bengio, and Geoffrey Hinton. Deep learning. *Nature*, 521(7553):436–444, 2015.
- [9] Joseph Redmon, Santosh Divvala, Ross Girshick, and Ali Farhadi. You only look once: Unified, real-time object detection. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 779–788, 2016.

- [10] Tomáš Křemen, Bruno Koska, Jiří Pospíšil, et al. Optimizing mobile laser scanning accuracy for urban applications: A comparison by strategy of different measured ground points. *Applied Sciences*, 14(8):3387, 2024.
- [11] Ultralytics. Yolo26 official documentation and architecture, 2026. Accessed: 2026-05-01.

Príloha A: obsah elektronickej prílohy

V elektronickej prílohe priloženej k práci sa nachádza zdrojový kód programu a súbory s výsledkami testovania. Zdrojový kód je zverejnený aj na stránke <https://github.com/DakotSvk/Bachelors-Thesis>.